Dizital Adda LMS

Production Completion Roadmap

Generated on 27/6/2026

Production Roadmap Summary

Goal: convert the current LMS prototype into a secure, reliable, production-ready LMS.

Fastest safe order: Authentication, Database, Course Management, Payment Verification, Enrollment, Learning Progress, Dashboards, Assignments, Quizzes, Certificates, Security, Testing, Deployment.

Estimated timeline: 8 weeks for a focused production push.

Phase 1: Stabilize Foundation

- Use one backend entrypoint. Merge backend/src/app.js and backend/src/server.js into one runtime server.
- Move all secrets to environment variables. Remove hardcoded JWT secrets, Razorpay test keys, and broken URLs.
- Create .env.example for backend and frontend.
- Create one frontend API client using VITE_API_URL.
- Fix malformed URLs like http://https://https://...
- Mount all required backend routes in the real server file.

Phase 2: Database First

- Add migration files for all production tables.
- Create users, courses, categories, sections, lectures, enrollments, orders, payments, progress, assignments, quizzes, certificates, reviews, notifications, attendance, and audit_logs.
- Add foreign keys between users, courses, enrollments, payments, lectures, and progress.
- Add unique constraints for user email, student ID, order ID, payment ID, and certificate code.
- Add created_at and updated_at fields to every major table.
- Seed one admin user safely using a hashed password.

Phase 3: Authentication And Security

- Fix JWT signing and verification to use the same JWT_SECRET.
- Hash every password, including student-created passwords.
- Protect every admin API with verifyToken and checkRole('admin').
- Protect teacher APIs with teacher ownership checks.
- Protect student APIs so students can only access their own data.
- Add validation middleware for login, courses, lectures, payments, assignments, and quizzes.
- Add Helmet, rate limiting, strict CORS, body limits, and centralized error handling.

Phase 4: Course, Section, Lecture System

- Complete course CRUD with title, description, price, category, teacher, thumbnail, status, duration, and level.
- Add publish/unpublish status for courses.

- Create sections linked to courses.
- Create lectures linked to sections.
- Upload videos and PDFs to Cloudinary or another production storage provider.
- Save secure video/PDF URLs in the lectures table.
- Block lecture access for users who are not enrolled.

Phase 5: Payment And Enrollment

- Create backend Razorpay order API.
- Store order records before checkout.
- Verify Razorpay signature on backend after payment.
- Add Razorpay webhook for reliable payment confirmation.
- Store payment ID, order ID, amount, currency, status, user ID, and course ID.
- Create enrollment only after verified payment success.
- Show payment history in student and admin portals.

Phase 6: Student Portal

- Show enrolled courses from real enrollments.
- Build continue learning from latest lecture progress.
- Add video player with lecture list and PDF notes.
- Add mark-as-complete API for lectures.
- Show course progress percentage.
- Show assignments, quizzes, certificates, and payment history.
- Allow student profile and password update.

Phase 7: Teacher Portal

- Show only courses assigned to the logged-in teacher.
- Allow teacher to create sections and lectures for own courses.
- Allow teacher to create assignments and quizzes.
- Show student submissions and quiz attempts.
- Allow grading and feedback.
- Allow teacher to reply to doubts.
- Prevent teacher from accessing another teacher's course data.

Phase 8: Admin Portal

- Create real admin dashboard analytics from database queries.
- Manage students, teachers, courses, categories, enrollments, and payments.
- Add block/unblock user controls.
- Add course approval and featured course controls.
- Add payment reports with date/status filters.
- Add CSV/PDF export from real backend data.
- Add audit logs for all admin actions.

Phase 9: Assignments, Quizzes, Certificates

- Assignments: teacher creates, student submits, teacher grades, student sees feedback.
- Quizzes: teacher creates quiz and questions, student attempts, backend calculates score.

- Progress: include lecture completion, assignment status, and quiz score.
- Certificates: generate only after payment verified and course requirements completed.
- Certificate verification: public verify endpoint and page using unique certificate code.

Phase 10: Production Quality

- Fix all ESLint errors.
- Remove unused imports, undefined handlers, static dashboard numbers, and debug alerts.
- Add backend tests for auth, RBAC, courses, payments, enrollments, and progress.
- Add frontend tests for login, protected routes, course purchase, and learning flow.
- Add structured logging and monitoring.
- Add database backup and restore strategy.
- Add CI pipeline for lint, build, and tests.

Recommended 8 Week Timeline

- Week 1: architecture cleanup, env, JWT, RBAC, database schema.
- Week 2: course, category, section, lecture, file upload.
- Week 3: Razorpay order, payment verification, webhooks, enrollment.
- Week 4: student dashboard, learning page, progress tracking.
- Week 5: teacher portal and admin portal real data.
- Week 6: assignments, quizzes, certificates.
- Week 7: validation, security hardening, tests, lint cleanup.
- Week 8: deployment, QA, monitoring, backups, final production checklist.

Immediate Top 20 Tasks

- 1. Merge backend entrypoints.
- 2. Fix JWT secret mismatch.
- 3. Protect admin routes.
- 4. Add database migrations.
- 5. Hash all passwords.
- 6. Fix frontend API URLs.
- 7. Create frontend API client.
- 8. Complete course CRUD.
- 9. Complete sections and lectures.
- 10. Add secure file upload.
- 11. Implement payment order storage.
- 12. Verify Razorpay signatures.
- 13. Create enrollment after verified payment.
- 14. Implement lecture progress updates.
- 15. Replace static dashboards with real APIs.
- 16. Complete teacher course ownership checks.
- 17. Complete assignments.
- 18. Complete quizzes.
- 19. Complete certificates.
- 20. Fix lint, tests, deployment, monitoring.